Computing (2025) 107:183
https://doi.org/10.1007/s00607-025-01540-w

REGULAR PAPER



A systematic mapping study on observability of microservices-based applications: fundamentals, classifications, and challenges

Francisco Gomes[1] · **Paulo Rego**[2] · **Fernando Trinta**[2]

Received: 21 November 2024 / Accepted: 5 August 2025 / Published online: 19 August 2025
© The Author(s), under exclusive licence to Springer-Verlag GmbH Austria, part of Springer Nature 2025

Abstract
Microservices-based applications are widely adopted for their scalability and support for continuous integration. While their modularity reduces development complexity, it increases operational and maintenance challenges, leading to more frequent and intricate system malfunctions. As cloud-native applications (CNAs) become increasingly sophisticated, traditional monitoring solutions prove inadequate, raising the risk of failures. These failures can include performance degradation, service outages, scalability limitations, and even potential data integrity. This has led to the concept of observability, defined as the ability to understand and diagnose a system's internal behavior by analyzing its external states. In CNAs, resource allocation is dynamic and dependent on factors like system health, making observability crucial not only for monitoring application health but also for triggering auto-scaling when necessary. Although some studies explore observability concepts, tools, and challenges, there is no comprehensive taxonomy. This paper proposes a taxonomy for observability in microservices-based applications, with a specific focus on auto-scaling, supported by a systematic mapping study (SMS). We analyze 66 studies published between 2019 and 2024, providing a thorough overview of observability. Additionally, the SMS highlights tools, benchmarking applications, real-world datasets, and ongoing issues in the field. This work aims to offer researchers an up-to-date and comprehensive resource for understanding observability in the context of CNAs.

Keywords Monitoring · Observability · Microservices · Taxonomy · Classifications · Challenges

Mathematics Subject Classification 68M14

Paulo Rego and Fernando Trinta contributed equally to this work.

Extended author information available on the last page of the article

1 Introduction

In recent years, software systems have increased in scale and complexity, making it more difficult to incorporate new functionalities efficiently. Microservices architecture emerged as a key solution, helping to avoid large monolithic codebases and enabling transparent application scaling [19]. Its benefits, such as improved performance and reduced development effort, have led to broad adoption, despite introducing new challenges for developers.

Cloud-native applications (CNAs) are designed to fully leverage cloud computing, using principles like containerization, microservices, dynamic orchestration, and continuous delivery. As CNAs become more distributed and complex, the risk of failure points and performance issues increases [53]. Diagnosis is challenging with hundreds of microservices running multiple instances [19], and traditional monitoring solutions can't provide full visibility or proactively detect issues. This creates a need for higher-level monitoring, known as observability [31], defined as the ability to infer a system's state from its outputs. Moreover, observability becomes crucial not only for understanding the health and behavior of applications but also for indicating when auto-scaling of resources is necessary [27, 43]. Auto-scaling refers to the process of dynamically adjusting the amount of computational resources allocated to an application based on current demand [18]. This adjustment is carried out without manual intervention, employing automated mechanisms to ensure that the application has adequate resources to operate efficiently and respond to fluctuations in its workload. Therefore, while microservices architecture facilitates scalable and flexible systems, it simultaneously highlights the importance of enhanced monitoring and elasticity to effectively manage and optimize the performance and reliability of CNAs.

To the best of our knowledge, a study that presents a comprehensive and up-to-date classification of observability in the context of microservices-based CNAs has not been published yet, particularly addressing aspects such as specific goals, integration, data collection methods, main sources, instrumentation practices, among other aspects. To address this research gap, a SMS of observability in microservices is necessary.

This work proposes a taxonomy for microservices-based applications, providing a structured approach to categorize the key aspects of observability. The main focus of this study is observability; however, we also aim to identify studies that relate this topic to auto-scaling, due to its relevance for improving application resources. The proposed taxonomy highlights key aspects of observability, which can help address the lack of a shared vocabulary in the field. This lack of a common understanding often leads to communication barriers, fragmented solutions, and inconsistencies in how tools and techniques are applied. A unified vocabulary enhances collaboration, ensures better alignment, and helps establish standardized best practices across all users of observability. The main contributions of this work are:

- A comprehensive taxonomy of observability in microservices-based applications, encompassing its main categories and enabling the classification of relevant studies; and

- An overview of tools, benchmarking applications, real-world datasets, and open challenges identified through the conducted SMS.

2 Background

This section covers key concepts, definitions, and characteristics essential for understanding the topic. Section 2.1 introduces the main concepts, Sect. 2.2 discusses key definitions from the literature, Sect. 2.3 outlines the benefits, and Sect. 2.4 explores the foundational pillars supporting the approach.

2.1 Introduction

In production environments, managing microservices requires ensuring smooth operation and promptly identifying failures or performance degradation. Both cloud service providers and users must continuously monitor resource usage, performance, and availability [34, 58]. In the event of failures or interruptions, monitoring data is crucial for diagnosing issues and facilitating repairs. The shift to the Cloud Native paradigm has increased operational complexity, making it harder to fully observe and understand application health, performance, and behavior. As a result, Observability emerged as a new strategy [46].

According to the Cloud Native Computing Foundation (CNCF) guide, one of the steps is Observability and Analysis [31]. Originating from Control Theory, observability defines a system as observable if its current state can be determined in finite time using only its outputs. Measuring microservice performance is crucial for enforcing quality of service (QoS) metrics, with the system needing to expose its state via instrumentation to meet service level agreement (SLA) parameters. Observability, composed of metrics, logs, and traces, is essential in cloud-native environments [31] to enhance understanding of applications. Niedermaier et al. [47] note that in cloud environments, observability reflects the degree to which infrastructure, applications, and their interactions can be monitored.

The motivations for adding observability to microservices-based applications stem from the execution phase [31]. These motivations include real-time tracking, system overhead measurement, and scalability, focusing on resource provisioning and scheduling. From a management perspective, key motivations involve root cause detection (anomalies, failure prediction, and dependency analysis), SLA management (load, availability, QoS, and violations), and end-to-end management (performance, response time, latency, and tracing).

2.2 Definitions of observability

Over the years, several authors have offered additional definitions of the concept of observability, complementing the ideas previously presented. According to Usman et al. [68], observability is an expanding set of practices combined with tools that go beyond mere monitoring, providing insights about the internal state of a system through the analysis of its external outputs, predominantly known as telemetry data,

such as metrics, logs, and traces. The proposed work adopts this definition, because it understands that it has a comprehensive scope on the topic and mentions the use of external outputs.

For Kratzke [32], observability represents the foundation for data-driven software development . Meanwhile, for Marie et al. [43], observability is an emerging concept that can be considered a broader set than monitoring, extending its boundaries and applying data analysis techniques.

2.3 Benefits of observability

The performance of microservices-based applications can be enhanced when supported by advanced observability capabilities [31]. Observability provides increased visibility into system activities, allowing teams to understand what happens in the system, where, how, and when. This clarity enables early detection of issues, whether performance-related or architectural, before they impact functionality. It also supports DevOps practices and the development process by offering real-time feedback that can guide design choices and trigger architectural refinements. Furthermore, observability contributes to system scalability by ensuring that monitoring components can handle the growth in data volume. The collected data not only enables automation, such as auto-scaling and failover, but also serves as input for deeper analysis using statistical methods and machine learning techniques. Ultimately, systems with robust observability tend to offer better user experiences, leading to greater customer satisfaction and supporting business success.

2.4 Types of observability data

As mentioned earlier, there are three main types of data often referred to as the pillars of observability [68]. They are: traces, metrics, and logs.

Traces offer a complete view of a request's flow through an application. A distributed trace consists of events generated at various points in a system, linked by a unique identifier. This identifier is passed through all components involved in fulfilling the request, connecting event data to the original request. Each trace represents a unique request, either synchronous (sequential) or asynchronous (parallel operations). Each operation in a trace is represented by a Span, a unit of work performed in the system.

Metrics provide insights into the state of a running system for developers and operators. They can be aggregated over time to identify trends and patterns, often visualized using various tools. Metrics cover a broad range, from low-level system data like CPU usage to high-level details like sales figures. They are essential for monitoring application health and triggering alerts for potential issues. While metrics are useful independently, correlating them with logs or traces offers a more comprehensive view of the system's state.

Logs record system activities and are crucial for debugging and issue investigation. In observability, logs complement traces and metrics by offering a detailed view of specific events. They are especially valuable for investigating real-time issues, as

they show what was happening in a system at a given moment, helping developers pinpoint failures or performance problems.

3 Related work

This section presents the related works to the current study, focusing on meta-studies about monitoring and observability approaches rather than primary research. These works represent the state of the art of this approach for microservices.

Costa et al. [16] propose a taxonomy for Fog monitoring solutions based on a systematic literature review and identify key challenges in the area. Studies published between 2012 and 2022 were analyzed, and ten were selected for classification according to the proposed taxonomy. The taxonomy encompasses characteristics such as purposes, topology, communication model, among others. The strengths and weaknesses of each solution were also assessed. Finally, the authors highlight the open challenges in the field.

Janes et al. [24] provide a critical comparison of 30 popular open-source tracing tools identified through a Multivocal Systematic Review. They classify tools based on factors like license, programming language, deployment, usage, collected data, and interoperability, emphasizing key aspects for operability, such as API availability, OpenTelemetry (OTel)[1] support, and self-hosting capability.

Giamattei et al. [21] conduct a grey literature review to identify, classify, and analyze monitoring tools for DevOps and microservices. The study examines 71 tools, providing an overview of available options and ongoing efforts to improve them. A map of the tools' characteristics was developed, covering their benefits, drawbacks, data collection techniques, and result presentation.

Niedermaier et al. [47] conduct a qualitative study to explore challenges and best practices in observability and monitoring of distributed systems. Through 28 semi-structured interviews with industry professionals, they identify key challenges, requirements, and solutions. The study highlights increasing complexity and the growing importance of observability for stable services and continuous application development. However, participants noted a gap in awareness of its importance among both management and developers. Additionally, they emphasized the need for an organizational strategy, roles, and responsibilities in implementing observability and monitoring.

Li et al. [35] survey microservices and observability, focusing on the complexity of industrial microservice systems on dynamic cloud infrastructures. They highlight challenges in diagnosing issues like failed requests and high latency due to numerous services and diverse environments. The study identifies new big data challenges in software engineering, with opportunities in adaptive log sampling, data fusion, smarter trace analysis, and business intelligence through trace analysis.

Usman et al. [68] conduct a survey on distributed IT environments and microservice observability, exploring challenges, requirements, and current solutions. They discuss essential functionalities for observability, open research questions on het-

[1] https://opentelemetry.io/.

erogeneous infrastructures, and the impact of data volume on system monitoring. The study also highlights the need for a unified platform for telemetry data and the influence of organizational culture and security concerns on the adoption of observability tools.

Kosińska et al. [31] present a systematic mapping of cloud-native application observability, focusing on engineering approaches, maturity, efficiency, tools, and future research directions. They analyze 56 studies from the past five years, covering traditional cloud and distributed computing as well as emerging areas like IoT, Edge, and Fog computing. The research highlights the intersection of observability with big data, security, and ML/AI, and also categorizes functional and non-functional requirements, containerization, orchestration, observability software, and data storage methods.

Table 1 compares the aforementioned studies. All studies presented some form of classification in monitoring and observability; however, none provided a taxonomy for observability in microservices-based applications. Additionally, none presented benchmarking applications or real-world workload datasets used in observability.

Table 1 Comparison of related works

Paper	Year	Period	Size	Context	Criteria	Type of study
Costa et al. [16]	2022	2012–2022	10	Fog monitoring	Context/taxonomy/challenges	Systematic review
Janes et al. [24]	2023	2022	30	Open-source tracing tools	Context/solutions	Systematic review
Giamattei et al. [21]	2023	2022	71	Monitoring tools	Context/solutions/challenges	Systematic review
Niedermaier et al. [47]	2019	2019	28	Observability and monitoring of distributed systems	Challenges/requirements/ solutions	Industry interviews
Li et al. [35]	2022	2022	25	Tracing and analysis of microservices	Context/challenges	Industry survey
Usman et al. [68]	2022	2015–2022	44	Observability of microservices-based on edge and containers	Context/tools/challenges	Survey
Kosinska et al. [31]	2023	2018–2022	56	Observability of CNA	Context/requirements/ tools/challenges	Systematic mapping
Our work	2024	2019–2024	66	Observability for microservices	Context/ taxonomy/ tools/ applications/ benchmarks/ datasets/ challenges	Systematic mapping

4 Research methodology

A preliminary study was conducted to identify existing secondary studies on observability, using Google Scholar to locate reviews, mapping studies, and surveys. This initial review highlighted the absence of a taxonomy like the one we propose, as well as the lack of consolidated information on tools, datasets, and evaluation practices, motivating the systematic mapping study (SMS) presented in this paper.

The methodology for this SMS is inspired by Petersen et al. [51]. The steps followed include scope definition, search, filtering, classification, and information extraction, culminating in the presentation of the results. The review process was initially conducted on March 1, 2024, and to complement the results, the same process was repeated on November 1, 2024.

Scope definition The first step is dedicated to defining the scope of the SMS, which contains the question that guides our research. The question sets a boundary for the scope of the work, i.e., the limit of the study area to be mapped. Subsequently, search strings are used to select articles. It is worth noting that before formulating the question, a consensus was reached to analyze topics useful for guiding research on "observability and auto-scaling in microservices". Therefore, this work proposes two Research Question (RQ):

- *RQ1 - What are the different domains and categories of observability in microservices?* This question aims to explore how observability benefits microservices-based applications and its relation to resource auto-scaling.
- *RQ2 - What are the different tools, applications benchmarks and dataset related to observability in microservices?* The objective of this question is to verify the tools, applications and dataset used in this context.

Search In this step, a search string was applied in four search engines: IEEE, ACM, Scopus, and Web of Science. These platforms are well-recognized in the academic community. Using multiple search engines ensures comprehensive coverage, including conference papers (IEEE and ACM) and scientific journals (Scopus and WoS). Scopus indexes databases such as Elsevier and Springer, while WoS covers over 9,000 journals. We used keywords most related to the theme "observability and auto-scaling in microservices" to generate the search string. After analyzing the terms, it was identified that the string "observability" would be necessary. The terms "monitoring and tracing" were used as a synonym to cover the general idea of observability. Additionally, the string "Microservice" needed to be included, with its plural form. The term "Auto-scaling" also needed to be included, along with synonyms such as "autoscaling, self-adaptive, self-scaling, elasticity". Finally, the string "observability" was added along with this last restriction to include all papers that focus on topics unrelated to auto-scaling. The final search string is shown in Fig. 1, along with the filtering process. In SCOPUS and WoS, the string was executed directly, while in IEEE and ACM, it was applied separately for terms in parentheses. The advanced search option was used to locate full-text works.

Filtering The filtering strategy considered direct exclusion criteria: (i) not written in English; (ii) fewer than six pages (short papers); (iii) a non-scientific article

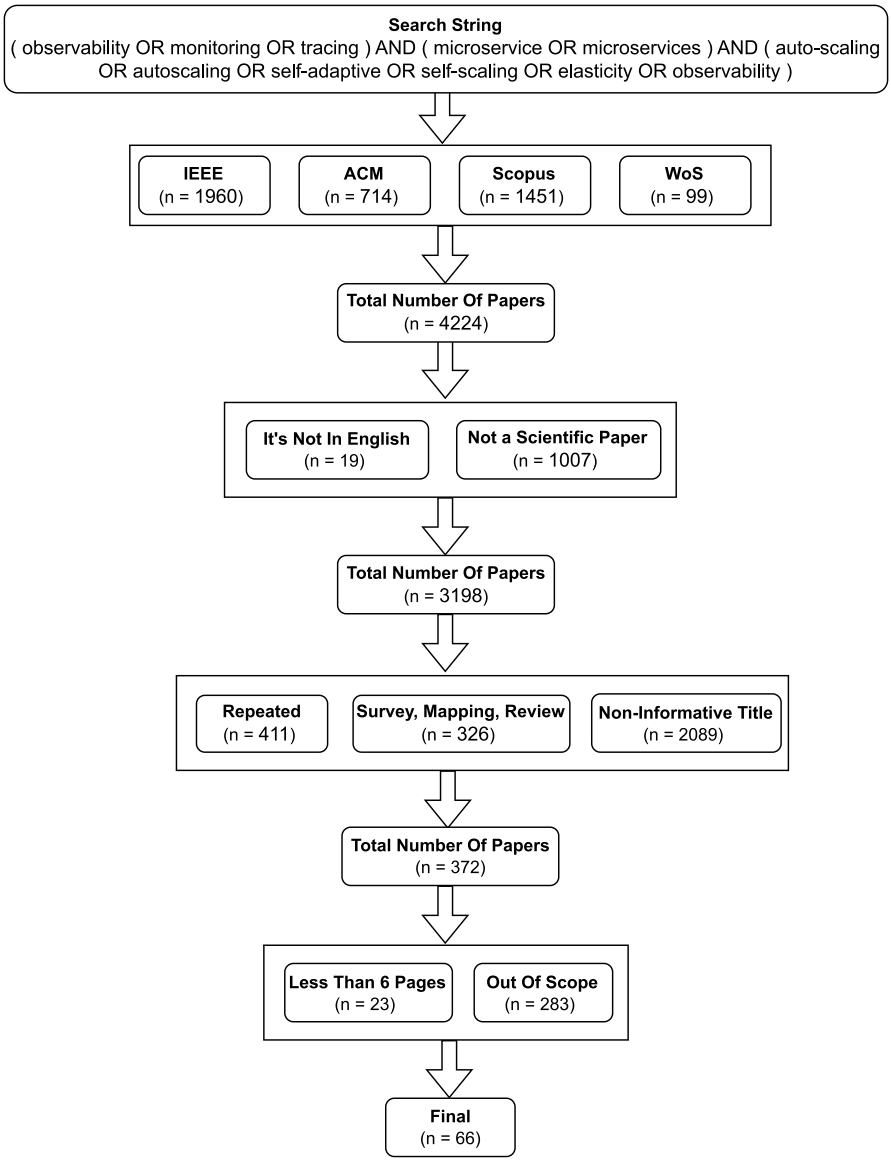


Fig. 1 Flowchart demonstrating the filtering process stages

(poster, tutorial, keynote, books, courses); (iv) a survey, mapping, or review; or (v) considered out of scope based on the title. Papers not meeting any exclusion criteria were included and advanced to the next stages. The inclusion criteria required that studies: (i) the full-text of the paper is available; (ii) the paper is a regular paper or a full paper; (iii) the paper is in the area of observability, monitoring e tracing; (iv) the paper is peer-reviewed (journal article, conference paper); and (v) the paper is between 2019–2024. This approach ensures that only relevant and sufficiently

detailed primary studies were considered. The flowchart in Fig. 1 shows the filtering process, with 4224 works and the number of works excluded at each stage. Thus, after applying all the criteria and checking for out-of-scope articles (last step), which also involves assessing the quality of the articles based on their full reading, 66 articles remained for classification.

Classification In the fourth step, we focused on classification, based on the reading of selected studies, categories were extracted to classify them and thus obtain more precise information. This process was divided into two phases. In the first phase, we defined main categories as well as the need for subcategories. In the second phase, we assigned classifications to each category and subcategory. As a result, the observability taxonomy for microservices-based applications was defined. (Sect. 5).

Information extraction In the fifth step, we conducted information extraction. For this study, we extracted information by directly reading the studies and mapping them according to the classifications. All the authors participated in the classification of the selected studies. The first author was responsible for the full reading of each of the identified studies, while the others followed the findings and guided the construction of the categories/subcategories, as well as the classification for positioning the selected studies. Additionally, the first author wrote the manuscript and condensed the results, and the others carried out the revisions. The methodological process, the selected studies, the taxonomy classifications, tools, benchmarking applications, and datasets can be viewed at the following link: https://doi.org/10.5281/zenodo.15715626.

5 Observability taxonomy

This section presents an observability taxonomy for microservices-based applications, derived from our SMS. It consolidates key domains and categories from the state of the art, summarized in Fig. 2, with detailed descriptions in the subsections. The development of the taxonomy was carried out in several stages: (i) it was initially based on the paper of Costa et al. [16]. After a full reading of the 66 studies, the first author identified the categories of purpose (Sect. 5.1), domains (Sect. 5.3), logs (Sect. 5.4), traces (Sect. 5.5), metrics (Sect. 5.6), and integration (Sects. 5.7); (ii) following the discovery of these categories/subcategories, the reviewers suggested reanalyzing each study to search for experimental elements, which led to the emergence of three additional categories: environment (Sect. 5.2), data collection (Sect. 5.8), and workload (Sect. 5.9); (iii) based on the identified categories, the reviewers recognized the need to further break down some of them, which led to the creation of subcategories; and (iv) the classification of categories and subcategories was based on their occurrence in the articles, with discussions among the reviewers to validate and refine the classification. The number of occurrences of each categorization from the selected studies is shown in parentheses.

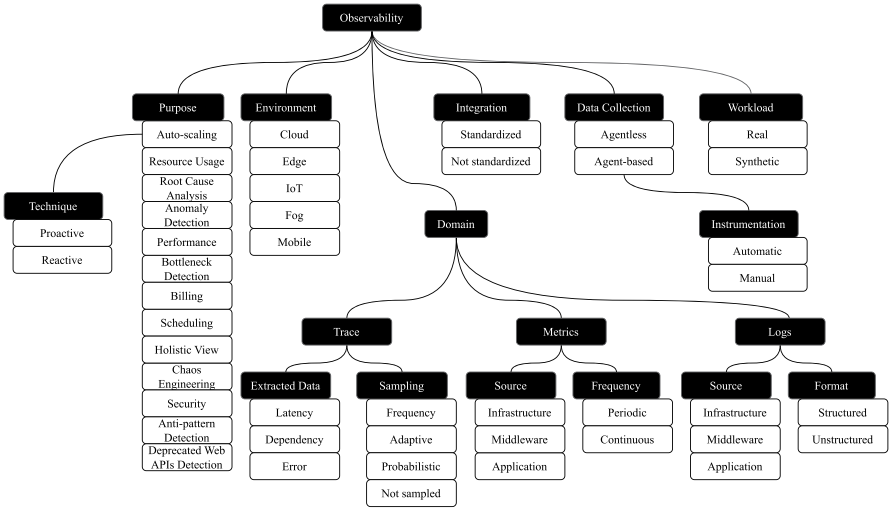


Fig. 2 Observability taxonomy

5.1 Purpose

This category outlines the main objectives of observability solutions as identified in the literature. These solutions monitor resource usage (2), such as CPU, memory, disk, and network, to ensure efficient allocation and detect bottlenecks (1), scheduling (4) and workload distribution to balance system demand. Anomaly detection (8) identifies unusual patterns or deviations from expected behavior. Observability also enables root cause analysis (RCA) (6) by tracing events and uncovering factors behind failures or performance issues. Monitoring performance (3) metrics ensures compliance with SLOs, while usage tracking supports billing (2), and cost optimization. A holistic view (11) of system components and their interactions improves overall understanding. Observability also supports chaos engineering (4), revealing system behavior under stress. It helps detect anti-patterns (1) in microservices and track deprecated APIs detection (1) to avoid future compatibility issues, and security (2) features enable threat detection, access tracking, and policy compliance. Finally, they support auto-scaling (22) by adjusting resources dynamically. There are two auto-scaling technique: Reactive (9) and Proactive (16). Reactive methods continuously monitor resource utilization and adjust allocation based on predefined thresholds. In contrast, proactive techniques predict resource demands and allocate resources in advance to accommodate anticipated workload changes.

5.2 Environment

Observability can be applied to various environments where microservices can be deployed, requiring customized approaches and specialized tools to address the unique characteristics of each environment: Cloud (64), Edge (3), Fog (1), IoT (3),

and Mobile (3). These environment classifications are based on explicit mentions in the selected studies, such as [17, 33, 67], and [29].

5.3 Domain

As discussed in Sect. 2.4, observability of distributed systems involves three main domains of instrumentation: Logs (23), Traces (49), and Metrics (58).

5.4 Logs

Logs are records that capture events and system states. This section presents the sources of log data collection and how logs are formatted.

5.4.1 Source

It refers to the log type. Infrastructure logs (9) capture system-level events such as CPU usage, disk activity, and network traffic, providing insights into the health and performance of hardware and virtual resources. Application logs (11) document software-level events, including user interactions, errors, and debugging information. Middleware logs (1) record activities from intermediary components such as message brokers, service meshes, and proxies, including message processing, routing events, and connection status.

5.4.2 Format

Log formats can be broadly categorized into structured and unstructured formats based on how the data is organized in log files. Structured logs (7) organize log data into a predefined schema or format, usually using key-value pairs or a structured data format like JSON, XML, or CSV. Unstructured logs (4) do not follow a predefined schema and instead store log data in a free-text, human-readable format.

5.5 Traces

Traces show the path of requests through a system. This section covers the extracted data and sampling process.

5.5.1 Extracted data

Through traces, valuable insights into the application can be obtained. Distributed tracing allows the measurement and analysis of latency (28), which is the time it takes for a request to traverse various components of a distributed system. It also provides visibility into the dependencies (24) between different services and components within the system, making it possible to understand the flow of data and control between services. Distributed tracing enables detection and diagnosis of errors (3) by identifying spans with exceptions, error codes, timeouts, or retries.

5.5.2 Sampling

Trace sampling reduces overhead in distributed systems. Frequency-based sampling (7) captures traces at fixed intervals, like every 1000 events. Probabilistic sampling (2) assigns a probability to each event, e.g., 0.1 for 10% capture. Adaptive sampling (1) dynamically adjusts the rate based on conditions like traffic thresholds. Some studies choose not to use sampling (2).

5.6 Metrics

Metrics are numerical representations of system behavior over time. This section presents the sources of metric data collection and their collection frequency.

5.6.1 Source

Metric levels refer to layers used to monitor a system's performance. Application metrics (26) track software behavior and performance, such as response time per endpoint and request count per route. Infrastructure metrics (50) monitor underlying components like CPU usage, memory, disk I/O, and network bandwidth. Middleware metrics (8) cover intermediary components such as message brokers, service meshes, and proxies, such as message throughput and request latency.

5.6.2 Frequency

There are two approaches to monitoring frequency: continuous and periodic. Continuous (2) monitoring involves the uninterrupted flow of monitoring data to the server once initiated. Periodic (21) monitoring allows for configuring a recurring period, with data being sent at regular intervals.

5.7 Integration

This category evaluates whether observability studies conform to a standardized data format. Some solutions use JSON files (3) as fundamental standardized data formats. In recent years, initiatives like OpenTelemetry (OTel) (22), which builds on OpenCensus[2] and OpenTracing[3] (5). The adoption of these standards facilitates data exchange and the integration of various monitoring components, even if they come from different vendors or operate at different application layers.

5.8 Data collection

The data generated by microservices is continuously collected by a centralized collector. Two main techniques are commonly used for this collection. The agent-based (38) approach involves collecting data from a service instance and then forwarding it

[2] https://opencensus.io/.

[3] https://opentracing.io/.

to the collector. In contrast, the agentless (1) approach allows the service instance to send its generated data directly to the collector.

5.8.1 Instrumentation

If collection is done by an agent to make a system observable, the system must be instrumented. Developers can instrument the code in two main ways. The manual (4) approach allows developers to gain deeper insights and gather telemetry directly from their applications. The automatic (10) approach is particularly useful for initial setups or in situations where modifying the application is not feasible. Both solutions offer comprehensive telemetry by extracting data either from the libraries used within the application or from the application's runtime environment.

5.9 Workload

A workload refers to the tasks or processes that a computing system needs to execute and is fundamental for testing and evaluating system performance. There are two main types of workloads used in benchmarks and performance tests: real workloads and synthetic workloads. Real workloads (21) are based on the actual operations a system performs in a production environment, representing the tasks and processes that end-users execute daily. In contrast, synthetic (47) workloads are artificially created to simulate the operations of a real system. These workloads are generated by benchmark tools and are designed to replicate specific usage patterns.

6 General classification

This section categorizes the selected observability studies for microservices-based applications according to the defined taxonomy of categories and subcategories. Table 2 lists the 66 selected studies along with their classification based on this taxonomy (each category has a value in parentheses for identification). In the Table, for the *Environment (1)*, entries are classified as Cloud (**C**), Edge (**E**), IoT (**I**), Fog (**F**), or Mobile (**M**). The *Domain (2)* refers to Metrics (**M**), Logs (**L**), or Traces (**T**). For *Purpose (3)*, entries are labeled as Resource Usage (**RU**), Bottlenecks (**B**), Scheduling (**Sched**), Anomaly Detection (**A**), Anti-patterns (**AP**), Performance (**P**), Billing (**Bill**), Security (**Sec**), Holistic View (**HV**), Auto-scaling (**AS**), Chaos Engineering (**CH**), or Deprecated Web APIs Detection (**DWAD**). For *Integration (4)*, **Y** indicates integration is used; **N** means it is not. In *Data Collection (5)*, **Y** indicates the use of an agent; **N** indicates otherwise. For *Instrumentation (6)*, entries are classified as Automatic (**Auto**) or Manual (**Man**). The *Workload (7)* is marked as Synthetic (**Synt**) or Real. For *Auto-scaling Technique (8)*, classifications are Proactive (**Pro**) or Reactive (**Reac**). For *Trace Sampling (9)*, options include Not Sampled (**Not**), Frequency-based (**Freq**), Adaptive (**Adap**), or Probabilistic (**Prob**). In *Trace Extracted Data (10)*, entries are Latency (**Lat**) or Dependency (**Dep**). For *Metrics Source (11)* and *Logs Source (14)*, sources are Infrastructure (**I**), Middleware (**M**) or Application (**A**). *Metrics Frequency (12)* is classified as Periodic (**Per**) or Continuous (**Cont**). *Logs*

Format (13) is either Structured (**S**) or Unstructured (**U**). If no value is identified, a dash (**-**) is used. For better table readability, here is the link with more details about the columns and values: https://doi.org/10.5281/zenodo.15715626.

According to the findings, observability in microservices-based applications serves various purposes, especially auto-scaling (33.33%) and holistic view (16.67%). Proactive auto-scaling techniques dominate (76.19%), and the Cloud is the main execution environment (96.96%). Traces are the primary data type used (74.24%), while only 28.78% of studies combine traces, metrics, and logs–highlighting the limited adoption of comprehensive observability. Some solutions adopt standards like OTel, OpenTracing, and JSON for integration; OTel appears in 22 studies and is cited in 8 more. Most studies use synthetic workloads (71.21%), and agents are commonly employed for data collection (57.57%). Instrumentation varies between automatic and manual approaches. Sampling is often used to reduce trace volume, mainly through frequency-based strategies. Trace data is typically used to extract latency and service dependencies (81.63%). Regarding metrics, infrastructure (e.g., CPU, memory), middleware (throughput) and application-level (e.g., response time) metrics are commonly used, with 21 out of 23 studies collecting metrics periodically. For logs, among 11 studies detailing format, 7 use structured and 4 unstructured logs; among 15 that specify the source, 5 use both infrastructure and application logs. All studies involve experimental evaluation. These results help answer **RQ1**, demonstrating diverse observability domains and categories, and reinforcing the need for a structured taxonomy to support future research.

The classification revealed important trends and gaps in observability for microservices-based applications. Traces are the most commonly used type of data, highlighting their relevance, and only 28.78% of the studies use traces, metrics, and logs together, indicating that comprehensive observability is still underexplored. Proactive auto-scaling techniques and synthetic workloads are predominant, suggesting a focus on controlled and predictive environments, which raises questions about applicability in real-world scenarios. The Cloud is the most frequently used environment, with little exploration of others. These observations reinforce the usefulness of the proposed taxonomy and highlight research gaps, which are further discussed as challenges in Sect. 9.

7 Datasets, benchmarking applications, and tools

This section presents the real-world workload datasets, benchmarking applications, and tools from the SMS conducted. The real-world workload datasets used in the selected studies are presented in Table 3. As previously mentioned in Sect. 5.9, employing real-world workloads is essential for evaluating and optimizing systems in environments that closely reflect their actual operational contexts.

A benchmarking application serves as a reference implementation to evaluate various aspects of microservices architectures. This type of application simulates a real-world scenario, typically involving multiple services that interact with each other to fulfill complex tasks. Table 4 shows the benchmarking applications used in the selected studies. The last row comprises unknown applications, which were either

Table 2 Selected studies

References	Year	1	2	3	4	5	6	7	8	9	10	11	12	13	14
[57]	2019	C	M-L-T	Sched	–	–	–	Synt	–	–	Top	I-A	–	–	A
[30]	2020	C	M-T	HV	Y	–	–	Real	Pro	–	Lat	I-A	–	–	–
[58]	2020	C	M-L-T	HV	Y	Y	–	Synt	–	Not	Lat	I	Per	S	–
[52]	2020	C	M	AS	–	–	–	Synt	Reac	–	–	I-A	Per	–	–
[15]	2020	C	M	AS	–	–	–	Synt	–	–	–	I	–	–	–
[42]	2020	C	M	AS	–	–	–	Synt	Pro	–	–	I-M	–	–	–
[76]	2020	C	M	AS	–	–	–	Synt	Pro	–	–	I-M	Per	–	–
[34]	2020	C	M	AS	–	Y	–	Synt	Reac	–	–	A	–	–	–
[72]	2021	C	M-L-T	RCA	–	Y	–	Real	–	–	Dep	I	–	–	A
[60]	2021	C	M	CH	–	–	–	Synt	–	–	–	I-A	–	–	–
[20]	2021	C	M-T	RCA	Y	Y	–	Synt	–	Freq	Dep	I	Per	–	–
[9]	2021	C	T	RU	Y	Y	–	Synt	–	–	Dep-Lat	–	–	–	–
[50]	2021	C	M-T	AS	–	–	–	Real	Pro	Freq	Dep-Lat	I	Per	–	–
[78]	2021	C	M-T	AS	–	Y	–	Synt	Pro	–	Dep	I	–	–	–
[4]	2021	C	M-T	AS	–	–	–	Real-Synt	Pro-Reac	–	Top	A	–	–	–
[3]	2021	C	M-T	AS	–	Y	–	Real	Reac	–	Dep	I-A	Per	–	–
[14]	2022	C	M-T	RU	–	–	–	Real	–	–	Top	I	–	–	–
[10]	2022	C-I	T	B	Y	Y	Auto	Synt	–	–	Dep-Lat	–	–	–	–
[32]	2022	C	M-L-T	HV	Y	Y	Man	Synt	–	Not	Run	I	–	S	I-A
[46]	2022	C-I	M-L-T	CH	–	–	–	Synt	–	–	Top	I-M	–	U	I
[79]	2022	C	M-T	Sched	–	Y	Auto	Synt	–	–	Top-Lat	I-M	–	–	–
[40]	2022	C	M-T	AS	–	–	–	Real	Pro	–	Dep	I	–	–	–
[67]	2022	C-I	M-L-T	AS	Y	Y	Auto	Synt	Pro	–	Lat	I-A	Cont	S	I-A
[71]	2022	C	M-T	AS	–	–	–	Real	Pro-Reac	–	Lat	I-A	–	–	–
[61]	2022	C-I	M-T	AS	–	–	–	Real-Synt	Reac	–	Dep	I	Per	–	–
[2]	2022	C	L	AP	Y	–	–	Synt	–	–	–	–	–	S	I-A
[54]	2023	C	M-L-T	A	–	Y	–	Real	–	–	Lat	I-A	–	–	A
[28]	2023	C	M	A	–	–	–	Synt	–	–	–	I-A	–	–	–

Table 2 (continued)

References	Year	1	2	3	4	5	6	7	8	9	10	11	12	13	14
[29]	2023	C-M	M-L	A	–	Y	–	Synt	–	–	–	I-A	–	–	I-A
[45]	2023	C	M	Sec	Y	Y	Man	Synt	–	Adap	–	–	–	–	–
[36]	2023	C	M-T	Sched	–	–	–	Synt	–	Prob	Dep	A	–	–	–
[77]	2023	C	M-L-T	RCA	Y	Y	Auto	Synt	–	–	Lat	I-A	Cont	U	A
[33]	2023	E	M-L-T	HV	Y	Y	–	Synt	–	–	Lat	I-A	–	–	A
[70]	2023	C	M-L-T	Bill	Y	Y	Auto	Synt	–	–	Run	I	Per	–	–
[69]	2023	E	M-L-T	HV	Y	Y	Auto	Synt	–	–	Run	I	–	U	–
[62]	2023	C	M-T	AS	–	–	–	Real	Pro	–	Dep	I-A	Per	–	–
[37]	2023	C-M	M-T	AS	–	–	–	Real	Pro	–	Dep	I	–	–	–
[8]	2023	C	M-T	AS	–	Y	–	Real	Pro	Freq	Dep	I-A	Per	–	–
[39]	2023	C	M-T	AS	Y	–	–	Real	Pro	Freq	Lat	I-A	–	–	–
[38]	2023	C-M	M-T	AS	Y	Y	–	Real	Pro	–	Lat	I-A	–	–	–
[44]	2023	C	M-T	AS	–	–	–	Synt	Pro	Freq	Dep	I-M	Per	–	–
[5]	2023	C	M-L-T	AS	–	–	–	Synt	Reac	–	Lat	I-A	Per	–	–
[22]	2023	C	M-L-T	RCA	Y	Y	–	Real	–	–	Dep-Error	I-A	–	–	A
[17]	2023	C-F	M-L-T	HV	Y	Y	–	Real	–	Freq	Dep-Lat	I-A	Per	–	I-A
[1]	2024	C-M	M	A	–	–	–	Synt	–	–	–	I	Per	–	–
[25]	2024	C	M-T	Sec	–	Y	–	Synt	–	–	Lat	I	–	–	–
[63]	2024	C	M	HV	Y	Y	–	Synt	–	–	–	I	Per	–	–
[13]	2023	C	M-T	P-Bill	Y	Y	–	Real	–	–	Lat	I-M	Per	–	–
[55]	2023	C	M-T	P	–	–	–	Synt	–	–	Lat	A	–	–	–
[26]	2023	C	M	HV	–	Y	–	Synt	–	–	–	I	–	–	–
[6]	2023	C	T	DWAD	Y	Y	Auto	Synt	–	–	Run	–	–	–	–
[65]	2023	C	M-L-T	A	Y	Y	–	Synt	–	–	Lat/ Error	A	–	–	–
[59]	2024	C	M-L-T	HV	Y	Y	Man	Synt	–	–	Lat	I	Per	–	–
[75]	2024	C	M	P	–	Y	–	Synt	–	–	–	I	Per	–	–
[48]	2024	C	M-L	A	–	N	–	Real	–	–	–	I-A	–	–	I

Table 2 (continued)

References	Year	1	2	3	4	5	6	7	8	9	10	11	12	13	14
[23]	2024	C	M-L-T	RCA	–	–	–	Real	–	–	Lat	I	–	S	–
[7]	2024	C	M-T	CH	Y	Y	Auto-Man	Synt	–	Prob	Lat	I-A	Per	–	–
[11]	2024	C	M-L	AS	Y	Y	–	Synt	Pro	–	–	I-M	Per	S	I-M
[66]	2024	C	T	HV	Y	–	–	Synt	–	–	Dep-Lat	–	–	–	–
[12]	2024	C	T	CH	Y	Y	Auto	Synt	–	–	Lat	–	–	–	–
[74]	2024	C	M	RCA	–	–	–	Real	–	–	–	–	–	–	–
[64]	2024	C-M	M-L-T	A	–	Y	–	Synt	–	–	Lat	I	–	S	I
[41]	2024	C-E	M-T	Sched	Y	Y	–	Synt	–	–	Lat	I-M	Per	–	–
[49]	2024	C	T	HV	Y	Y	Auto	Synt	–	–	Lat	–	–	–	–
[56]	2024	C	M-L-T	A	–	Y	–	Synt	–	–	Top	–	–	U	–
[73]	2024	C	T	AS	Y	Y	–	Synt	–	–	Dep-Lat	–	–	–	–

Table 3 Distribution of real-world workload datasets

Name	Paper References	Available	Size
Wikipedia	[3, 4, 62, 71]	http://www.wikibench.eu/?page_id=60	171.6 GB
WorldCup98	[61, 71]	https://ita.ee.lbl.gov/html/contrib/WorldCup.html	125 GB
AzurePublicDatasetV2	[37, 50]	https://github.com/Azure/AzurePublicDataset	235 GB
Facebook	[13, 14]	https://networkrepository.com/socfb.php	376 MB
INRIA	[13, 14]	https://project.inria.fr/aeriallimagelabeling/	19.5 GB
Alibaba	[40, 73]	https://github.com/alibaba/clusterdata/tree/master/cluster-trace-microservices-v2022	2 TB
AIOps	[23, 54]	https://github.com/NetManAIOps/AIOps-Challenge-2020-Data	2.9 GB
AOL Query Logs	[61]	https://www.kaggle.com/datasets/dineshydv/aol-user-session-collection-500k	226.55 MB
ClarkNet	[8]	https://ita.ee.lbl.gov/html/contrib/ClarkNet-HTTP.html	172.5 MB
Twitter	[4]	https://archive.org/details/twitterstream	6.8 TB
WITS	[62]	https://wand.net.nz/wits/	65 GB

Table 4 Distribution of benchmarking application

Benchmarking application	Paper References
Social network	[3, 4, 8, 13, 14, 20, 23, 36, 38–40, 50, 52, 62, 66, 78]
Online boutique	[3, 9, 37, 42, 44, 50, 62, 66, 73, 74, 77]
Hotel reservation	[4, 12–14, 20, 36, 40, 52, 78]
Train-ticket	[2, 3, 12, 44, 45, 52, 54, 66, 77]
Hipster shop	[12, 23, 34, 54, 61, 76]
Media service	[4, 20, 40, 52, 62]
Sock shop	[30, 38, 39, 57]
OpenTelemetry demo	[7, 59, 65]
BookInfo	[44, 60]
Unknown	[1, 5, 6, 10, 11, 15, 17, 22, 25, 26, 28, 29, 32, 33, 41, 46, 48, 49, 55, 56, 58, 63, 64, 67, 69–72, 75, 79]

not identified by the authors of this work or are proprietary. An overview of the tools used in the selected studies is presented in Table 5. These tools focus on load testing, monitoring, distributed tracing, logging, visualization, and service mesh.

These findings contribute to answering **RQ2**, presenting the tools, applications, and datasets used in this observability context. The analysis of real-world workload datasets, benchmarking applications, and tools in the selected studies reveals a vari-

Table 5 Tools used in the selected studies

Type	Name	Paper References
Load testing	Locust	[3, 7, 12–14, 34, 37, 41, 42, 44, 50, 59, 71, 73, 78]
	Wrk2	[20, 36, 52]
	JMeter	[79]
	Gatling	[45]
	Vegeta	[67]
Monitoring	Prometheus	[1, 3, 4, 7, 13–15, 17, 20, 29, 30, 33, 37, 40–42, 44, 46, 48, 50, 52, 58–63, 67, 69, 71, 74, 76, 77]
	cAdvisor	[1, 13, 17, 20, 28, 30, 37–39, 41, 44, 50, 52, 58, 60–62, 77]
	Elastic beats	[17, 32, 70, 74]
	Heapster	[5, 57]
	BCC	[34]
Distributed tracing	Jaeger	[7–10, 12–14, 17, 20, 33, 38, 40, 41, 44, 50, 52, 58, 69, 73, 78]
	Zipkin	[5, 57, 67]
Logging	Elasticsearch	[5, 12, 17, 32, 69, 70, 73]
	Loki	[28, 33, 69, 77]
	Fluentd	[58, 67]
	Logstash	[5, 33]
Visualization	Grafana	[17, 29, 30, 33, 34, 46, 48, 59, 60, 63, 67, 69]
	Kibana	[32, 70]
Service mesh	Istio	[2, 12, 13, 41, 44, 46, 61, 76]
	Linkerd	[37, 42, 50]

ety of approaches in the context of observability in microservices architectures. The datasets differ greatly in type and size, highlighting the broad range of scenarios where observability is applied. The benchmarking applications also show diversity, with social networks and e-commerce scenarios being particularly popular. The tools used focus on key observability areas, but a lack of standardization in tools and datasets limits direct comparisons between studies. This indicates that future research should work toward better practices for integrating and adapting tools and datasets more effectively.

8 Threats to validity

This section identifies potential threats to the study's validity, categorized into construct, internal, external, and conclusion validity.

Construct validity The research questions may not fully capture all aspects of observability in microservices. Although the taxonomy was based on recurring elements from the literature, emerging or less discussed aspects might have been overlooked, potentially limiting the classification's comprehensiveness.

Internal validity To minimize bias, a predefined search string was used across four major engines. Inclusion and exclusion criteria were clearly defined, and the selection process was reviewed. However, some relevant studies may have been missed due to terminology variations or indexing issues.

External validity The study's findings are based on 66 studies published between 2019 and 2024, which may exclude earlier foundational works or very recent, unindexed publications.

Conclusion validity Potential threats include biases in data extraction, subjective classifications, and dataset scope limitations. To mitigate these, we followed a structured review protocol and provide a public dataset to ensure transparency, repeatability, and enable replication or extension.

9 Challenges and future directions

This section presents the research challenges and future trends in the Observability.

Metametric insights This challenge arises because basic collected metrics do not fully capture the complex behavior of microservices. There is a lack of integrated methods to correlate and process metrics to create more precise indicators of system state.

Automatically detect anti-patterns Due to the growing complexity of microservices architectures and the volume of generated data, manual detection of problematic patterns becomes unfeasible. Automating this detection is essential to maintain quality and performance.

Catalog of observability anti-patterns Without consolidated documentation of common anti-patterns, teams tend to repeat known mistakes, compromising system reliability. The absence of such a catalog limits learning and effective prevention.

Lack of tool and dataset standardization The lack of clear standardization in observability tools and datasets hinders direct comparisons between studies and can slow progress in the field. This challenge arises mainly because the observability field is rapidly evolving, with many vendors and open-source tools adopting different data formats and metrics. The absence of unified standards creates fragmentation, making it difficult to compare studies or integrate solutions seamlessly.

Incorporating all three data types Although traces, metrics, and logs are complementary, their technical integration remains challenging. Many solutions focus only on traces, which limits the comprehensive view that could be achieved by combining all three.

Exploration of observability in diverse environments Most research focuses on cloud environments, but there is a growing diversity of environments (Edge, Fog, IoT, and Mobile) with specific characteristics. Exploring observability in diverse contexts can offer valuable insights and enhance the applicability of solutions. The lack of focus on these environments limits solution applicability.

Proactive auto-scaling and synthetic workloads Many observability solutions rely on synthetic workloads for evaluation, as real-world workloads are often hard to access or reproduce. However, synthetic scenarios may not reflect the complexity and variability of real systems, limiting the applicability of findings. Additionally,

proactive auto-scaling remains challenging because it requires accurate forecasting and timely reactions to dynamic, often unpredictable workload patterns–conditions that are difficult to simulate and observe reliably.

10 Conclusion

The increasing complexity and scale of cloud-native applications, particularly those based on microservices, have brought new challenges for monitoring and performance management. In this context, observability has emerged as a crucial capability to ensure system transparency, reliability, and responsiveness to dynamic workloads. This study addressed the need for a more structured understanding of observability in microservices-based cloud-native applications by proposing a comprehensive taxonomy that organizes the most relevant domains and categories in the field. Through a SMS, we identified and classified existing solutions according to the proposed taxonomy, revealing the diversity of approaches and the fragmentation in how observability is currently addressed. In addition, by mapping tools, benchmarking applications, datasets, and open challenges and future directions. As future work, we aim to (i) update and expand the taxonomy with recent studies and tools; and (ii) explore its use in Site Reliability Engineering, particularly for defining SLIs/SLOs, improving alerting, incident response, and task prioritization.

Acknowledgements This study was partially supported by the Brazilian Federal Agency for Support and Evaluation of Graduate Education (CAPES) under the Grant Number 001.

Author contributions Francisco Gomes wrote the main manuscript text and prepared figures. Paulo Rego and Fernando Trinta reviewed the manuscript.

Data availability No datasets were generated or analysed during the current study.

Declarations

Conflict of interest The authors declare no Conflict of interest.

References

1. Akbari M, Bolla R, Bruschi R, Davoli F, Lombardo C, Siccardi B (2024) A monitoring, observability and analytics framework to improve the sustainability of B5G technologies. In: 2024 IEEE international conference on communications workshops (ICC workshops), pp 969–975. IEEE
2. Al Maruf A, Bakhtin A, Cerny T, Taibi D (2022) Using microservice telemetry data for system dynamic analysis. In: 2022 IEEE international conference on service-oriented system engineering (SOSE), pp 29–38. IEEE
3. Baarzi AF, Kesidis G (2021) Showar: Right-sizing and efficient scheduling of microservices. In: Proceedings of the ACM symposium on cloud computing, pp 427–441
4. Bhasi VM, Gunasekaran JR, Thinakaran P, Mishra CS, Kandemir MT, Das C (2021) Kraken: Adaptive container provisioning for deploying dynamic DAGs in serverless platforms. In: Proceedings of the ACM symposium on cloud computing, pp 153–167

5. Bi Q, Liu Y, Zhang L, Kang K, Yang D, Min S (2023) Dynamic scalability mechanisms for microservices in federated cloud platform. In: 2023 IEEE 5th international conference on civil aviation safety and information technology (ICCASIT), pp 732–737. IEEE
6. Bonorden L, Hoorn A (2024) Detecting usage of deprecated web APIs via tracing. In: 2024 IEEE 21st international conference on software architecture (ICSA), pp 23–33. https://doi.org/10.1109/ICSA59870.2024.00011
7. Borges MC, Bauer J, Werner S, Gebauer M, Tai S (2024) Informed and assessable observability design decisions in cloud-native microservice applications. arXiv preprint arXiv:2403.00633
8. Cai B, Wang B, Yang M, Guo Q (2023) Automan: resource-efficient provisioning with tail latency guarantees for microservices. Futur Gener Comput Syst 143:61–75
9. Cassé C, Berthou P, Owezarski P, Josset S (2021) Using distributed tracing to identify inefficient resources composition in cloud applications. In: 2021 IEEE 10th international conference on cloud networking (CloudNet), pp 40–47. IEEE
10. Cassé C, Berthou P, Owezarski P, Josset S (2022) A tracing based model to identify bottlenecks in physically distributed applications. In: 2022 international conference on information networking (ICOIN), pp 226–231. IEEE
11. Chakraborty A, Eswaran A, Thorat P, Verma M, Gupta P, Jayachandran P (2024) Intent-driven multi-engine observability dataflows for heterogeneous geo-distributed clouds. In: 2024 IEEE 17th international conference on cloud computing (CLOUD), pp 30–41. IEEE
12. Chen H, Chen P, Yu G, Li X, He Z, Zhang H (2024) Microfi: Non-intrusive and prioritized request-level fault injection for microservice applications. IEEE Trans Depend Secure Comput
13. Chow K-H, Deshpande U, Deenadayalan V, Seshadri S, Liu L (2024) Atlas: hybrid cloud migration advisor for interactive microservices. In: Proceedings of the Nineteenth European conference on computer systems, pp 870–887
14. Chow K-H, Deshpande U, Seshadri S, Liu L (2022) Deeprest: deep resource estimation for interactive microservices. In: Proceedings of the seventeenth European conference on computer systems, pp 181–198
15. Chowdhury R, Talhi C, Ould-Slimane H, Mourad A (2020) A framework for automated monitoring and orchestration of cloud-native applications. In: 2020 international symposium on networks, computers and communications (ISNCC), pp 1–6. IEEE
16. Costa B, Bachiega J Jr, Carvalho LR, Rosa M, Araujo A (2022) Monitoring fog computing: a review, taxonomy and open challenges. Comput Netw 215:109189
17. Costa B, Banerjee A, Jayaraman PP, Carvalho LR, Bachiega Jr, J, Araujo A (2023) Achieving observability on fog computing with the use of open-source tools. In: International conference on mobile and ubiquitous systems: computing, networking, and services, pp 319–340. Springer
18. Dogani J, Namvar R, Khunjush F (2023) Auto-scaling techniques in container-based cloud and edge/fog computing: taxonomy and survey. Comput Commun
19. Dragoni N, Giallorenzo S, Lafuente AL, Mazzara M, Montesi F, Mustafin R, Safina L (2017) Microservices: yesterday, today, and tomorrow. Present and ulterior software engineering, pp 195–216
20. Gan Y, Liang M, Dev S, Lo D, Delimitrou C (2021) Sage: practical and scalable ML-driven performance debugging in microservices. In: Proceedings of the 26th ACM international conference on architectural support for programming languages and operating systems, pp 135–151
21. Giamattei L, Guerriero A, Pietrantuono R, Russo S, Malavolta I, Islam T, Dînga M, Koziolek A, Singh S, Armbruster M, et al (2023) Monitoring tools for DevOps and microservices: a systematic grey literature review. J Syst Softw, 111906
22. Gu S, Rong G, Ren T, Zhang H, Shen H, Yu Y, Li X, Ouyang J, Chen C (2023) TrinityRCL: multi-granular and code-level root cause localization using multiple types of telemetry data in microservice systems. IEEE Trans Software Eng 49(5):3071–3088
23. Han Y, Du Q, Huang Y, Li P, Shi X, Wu J, Fang P, Tian F, He C (2024) Holistic root cause analysis for failures in cloud-native systems through observability data. IEEE Trans Serv Comput
24. Janes A, Li X, Lenarduzzi V (2023) Open tracing tools: Overview and critical comparison. J Syst Softw, 111793
25. Kahlhofer M, Rass S (2024) Application layer cyber deception without developer interaction. arXiv preprint arXiv:2405.12852
26. Kannan PG, Gupta SM, Behl D, Raichstein E, Takvorian J (2024) Designing a lightweight network observability agent for cloud applications. In: International conference on passive and active network measurement, pp 262–276. Springer

27. Karumuri S, Solleza F, Zdonik S, Tatbul N (2021) Towards observability data management at scale. ACM SIGMOD Rec 49(4):18–23
28. Kawasaki J, Koyama D, Miyasaka T, Otani T (2023) Failure prediction in cloud native 5G core with eBPF-based observability. In: 2023 IEEE 97th vehicular technology conference (VTC2023-Spring), pp 1–6. IEEE
29. Khichane A, Fajjari I, Aitsaadi N, Gueroui M (2023) 5GC-observer: a non-intrusive observability framework for cloud native 5G system. In: NOMS 2023-2023 IEEE/IFIP network operations and management symposium, pp 1–10. IEEE
30. Kosińska J, Zieliński K (2020) Autonomic management framework for cloud-native applications. J Grid Comput 18:779–796
31. Kosińska J, Baliś B, Konieczny M, Malawski M, Zieliński S (2023) Towards the observability of cloud-native applications: the overview of the state-of-the-art. IEEE Access
32. Kratzke N (2022) Cloud-native observability: the many-faceted benefits of structured and unified logging-a multi-case study. Future Internet 14(10):274
33. Kumar A, Ahmed T, Saini K, Kumar J (2023) Neos: Non-intrusive edge observability stack based on zero trust security model for ubiquitous computing. In: 2023 IEEE international conference on edge computing and communications (EDGE), pp 79–84. IEEE
34. Levin J, Benson TA (2020) Viperprobe: Rethinking microservice observability with ebpf. In: 2020 IEEE 9th international conference on cloud networking (CloudNet), pp 1–8. IEEE
35. Li B, Peng X, Xiang Q, Wang H, Xie T, Sun J, Liu X (2022) Enjoy your observability: an industrial survey of microservice tracing and analysis. Empir Softw Eng 27:1–28
36. Li X, Zhou J, Wei X, Li D, Qian Z, Wu J, Qin X, Lu S (2023) Topology-aware scheduling framework for microservice applications in cloud. IEEE Trans Parallel Distrib Syst 34(5):1635–1649
37. Liao J, Zhou Z, Xu F, Chen X (2023) Staaf: Spatial-temporal correlations aware autoscaling framework for microservices. In: 2023 IEEE smart world congress (SWC), pp 1–9. IEEE
38. Liu J, Wang Q, Zhang S, Hu L, Da Silva D (2023) Sora: a latency sensitive approach for microservice soft resource adaptation. In: Proceedings of the 24th international middleware conference, pp 43–56
39. Liu J, Zhang S, Wang Q (2023) μconadapter: Reinforcement learning-based fast concurrency adaptation for microservices in cloud. In: Proceedings of the 2023 ACM symposium on cloud computing, pp 427–442
40. Luo S, Xu H, Ye K, Xu G, Zhang L, Yang G, Xu C (2022) The power of prediction: microservice auto scaling via workload learning. In: Proceedings of the 13th symposium on cloud computing, pp 355–369
41. Marchese A, Tomarchio O (2024) Telemetry-driven microservices orchestration in cloud-edge environments. In: 2024 IEEE 17th international conference on cloud computing (CLOUD), pp 91–101. IEEE
42. Marie-Magdelaine N, Ahmed T (2020) Proactive autoscaling for cloud-native applications using machine learning. In: GLOBECOM 2020-2020 IEEE global communications conference, pp 1–7. IEEE
43. Marie-Magdelaine N, Ahmed T, Astruc-Amato G (2019) Demonstration of an observability framework for cloud native microservices. In: 2019 IFIP/IEEE symposium on integrated network and service management (IM), pp 722–724. IEEE
44. Meng C, Song S, Tong H, Pan M, Yu Y (2023) Deepscaler: holistic autoscaling for microservices based on spatiotemporal GNN with adaptive graph learning. In: 2023 38th IEEE/ACM international conference on automated software engineering (ASE), pp 53–65. IEEE
45. Monteiro D, Yu Y, Zisman A, Nuseibeh B (2023) Adaptive observability for forensic-ready microservice systems. IEEE Trans Serv Comput
46. Naqvi MA, Malik S, Astekin M, Moonen L (2022) On evaluating self-adaptive and self-healing systems using chaos engineering. In: 2022 IEEE international conference on autonomic computing and self-organizing systems (ACSOS), pp 1–10. IEEE
47. Niedermaier S, Koetter F, Freymann A, Wagner S (2019) On observability and monitoring of distributed systems–an industry interview study. In: Service-oriented computing: 17th international conference, ICSOC 2019, Toulouse, France, October 28–31, 2019, Proceedings 17, pp 36–52. Springer
48. Noetzold D, Rossetto AG, Leithardt VR, Costa HDM (2024) Enhancing infrastructure observability: machine learning for proactive monitoring and anomaly detection
49. Otero M, Garcia JM, Fernandez P (2024) Towards a lightweight distributed telemetry for microservices. In: 2024 IEEE 44th international conference on distributed computing systems workshops (ICDCSW), pp 75–82. IEEE

50. Park J, Choi B, Lee C, Han D (2021) Graf: A graph neural network based proactive resource allocation framework for slo-oriented microservices. In: Proceedings of the 17th international conference on emerging networking experiments and technologies, pp 154–167
51. Petersen K, Feldt R, Mujtaba S, Mattsson M (2008) Systematic mapping studies in software engineering. In: 12th international conference on evaluation and assessment in software engineering (EASE). BCS learning & development
52. Qiu H, Banerjee SS, Jha S, Kalbarczyk ZT, Iyer RK (2020) Firm: an intelligent fine-grained resource management framework for slo-oriented microservices. In: 14th USENIX symposium on operating systems design and implementation (OSDI 20), pp 805–825
53. Razzaq A, Ghayyur SA (2023) A systematic mapping study: the new age of software architecture from monolithic to microservice architecture-awareness and challenges. Comput Appl Eng Educ 31(2):421–451
54. Ren R, Wang Y, Liu F, Li Z, Xie G (2023) Triple: the interpretable deep learning anomaly detection framework based on trace-metric-log of microservice. In: 2023 IEEE/ACM 31st international symposium on quality of service (IWQoS), pp 1–10. IEEE
55. Rezvani M, Jahanshahi A, Wong D (2024) Characterizing in-kernel observability of latency-sensitive request-level metrics with ebpf. In: 2024 IEEE international symposium on performance analysis of systems and software (ISPASS), pp 24–35. IEEE
56. Saha A, Agarwal P, Ghosh S, Gantayat N, Sindhgatta R (2024) Towards business process observability. In: Proceedings of the 7th joint international conference on data science & management of data (11th ACM IKDD CODS and 29th COMAD), pp 257–265
57. Sampaio AR, Rubin J, Beschastnikh I, Rosa NS (2019) Improving microservice-based applications with runtime placement adaptation. J Internet Serv Appl 10:1–30
58. Scrocca M, Tommasini R, Margara A, Valle ED, Sakr S (2020) The kaiju project: enabling event-driven observability. In: Proceedings of the 14th ACM international conference on distributed and event-based systems, pp 85–96
59. Sharma B, Nadig D (2024) ebpf-enhanced complete observability solution for cloud-native microservices. In: ICC 2024-IEEE international conference on communications, pp 1980–1985. IEEE
60. Simonsson J, Zhang L, Morin B, Baudry B, Monperrus M (2021) Observability and chaos engineering on system calls for containerized applications in docker. Futur Gener Comput Syst 122:117–129
61. Song Y, Li C, Zhuang K, Ma T, Wo T (2022) An automatic scaling system for online application with microservices architecture. In: 2022 IEEE international conference on joint cloud computing (JCC), pp 73–78. IEEE
62. Son M, Mohanty S, Gunasekaran JR, Kandemir M (2023) Microblend: an automated service-blending framework for microservice-based cloud applications. In: 2023 IEEE 16th international conference on cloud computing (CLOUD), pp 460–470. IEEE
63. Sreemathy S, Kumar V, Priyadharshini S (2024) Application monitoring and telemetry analytics. In: 2024 7th international conference on circuit power and computing technologies (ICCPCT), vol 1, pp 1559–1565. IEEE
64. Sundqvist T, Bhuyan M, Elmroth E (2023) Robust procedural learning for anomaly detection and observability in 5G RAN. IEEE Trans Netw Serv Manag
65. Talaver OV, Vakaliuk TA (2023) Dynamic system analysis using telemetry. In: CS &SE@ SW, pp 193–209
66. Tian X, Ying S, Li T, Yuan M, Wang R, Zhao Y, Shang J (2024) iTCRL: causal-intervention-based trace contrastive representation learning for microservice systems. IEEE Trans Softw Eng
67. Tzanettis I, Androna C-M, Zafeiropoulos A, Fotopoulou E, Papavassiliou S (2022) Data fusion of observability signals for assisting orchestration of distributed applications. Sensors 22(5):2061
68. Usman M, Ferlin S, Brunstrom A, Taheri J (2022) A survey on observability of distributed edge & container-based microservices. IEEE Access 10:86904–86919
69. Usman M, Ferlin S, Brunstrom A, Taheri J (2023) Desk: Distributed observability framework for edge-based containerized microservices. In: 2023 Joint European conference on networks and communications & 6G summit (EuCNC/6G Summit), pp 617–622. IEEE
70. Vries S, Blaauw F, Andrikopoulos V (2023) Cost-profiling microservice applications using an APM stack. Future Internet 15(1):37
71. Vu D-D, Tran M-N, Kim Y (2022) Predictive hybrid autoscaling for containerized applications. IEEE Access 10:109768–109778

72. Wang H, Wu Z, Jiang H, Huang Y, Wang J, Kopru S, Xie T (2021) Groot: An event-graph-based approach for root cause analysis in industrial settings. In: 2021 36th IEEE/ACM international conference on automated software engineering (ASE), pp 419–429. IEEE
73. Xie R, Wang L, Song C (2024) Towards minimum latency in cloud-native applications via service-characteristic-aware microservice deployment. In: 2024 IEEE international conference on software analysis, evolution and reengineering (SANER), pp 35–46. IEEE
74. Xie Z, Zhang S, Geng Y, Zhang Y, Ma M, Nie X, Yao Z, Xu L, Sun Y, Li W, et al (2024) Microservice root cause analysis with limited observability through intervention recognition in the latent space. In: Proceedings of the 30th ACM SIGKDD conference on knowledge discovery and data mining, pp 6049–6060
75. Yokelson D, Titov M, Ramesh S, Kilic O, Turilli M, Jha S, Malony A (2024) Enabling performance observability for heterogeneous HPC workflows with soma. In: Proceedings of the 53rd international conference on parallel processing, pp 220–230
76. Yu G, Chen P, Zheng Z (2020) Microscaler: cost-effective scaling for microservice applications in the cloud with an online learning approach. IEEE Trans Cloud Comput 10(2):1100–1116
77. Yu G, Chen P, Li Y, Chen H, Li X, Zheng Z (2023) Nezha: interpretable fine-grained root causes analysis for microservices on multi-modal observability data. In: Proceedings of the 31st ACM joint European software engineering conference and symposium on the foundations of software engineering, pp 553–565
78. Zhang Y, Hua W, Zhou Z, Suh GE, Delimitrou C (2021) Sinan: Ml-based and qos-aware resource management for cloud microservices. In: Proceedings of the 26th ACM international conference on architectural support for programming languages and operating systems, pp 167–181
79. Zhu J, Yang R, Sun X, Wo T, Hu C, Peng H, Xiao J, Zomaya AY, Xu J (2022) QoS-Aware Co-scheduling for distributed long-running applications on shared clusters. IEEE Trans Parallel Distrib Syst 33(12):4818–4834

Publisher's Note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.

Springer Nature or its licensor (e.g. a society or other partner) holds exclusive rights to this article under a publishing agreement with the author(s) or other rightsholder(s); author self-archiving of the accepted manuscript version of this article is solely governed by the terms of such publishing agreement and applicable law.

Authors and Affiliations

Francisco Gomes[1] · Paulo Rego[2] · Fernando Trinta[2]

✉ Francisco Gomes
almada@crateus.ufc.br

Paulo Rego
paulo@dc.ufc.br

Fernando Trinta
fernando.trinta@dc.ufc.br

[1] Federal University of Ceará, Crateús, Ceará 63700-000, Brazil

[2] Federal University of Ceará, Fortaleza, Ceará 60020-181, Brazil